Proyecto FHM #01

Localización bayesiana de un robot autónomo
Implementa un filtro de Bayes discreto desde cero

Objetivo: diseñar, programar y evaluar un estimador que deduzca dónde está un robot móvil utilizando órdenes de movimiento imperfectas y mediciones ruidosas. No se permite utilizar librerías que implementen el filtro.

Nivel: intermedio (probabilidad + Python/C++). Tiempo orientativo: 8–15 horas. Producto final: repositorio reproducible, pruebas y breve informe técnico.

1. Historia del problema

Un robot terrestre se desplaza a lo largo de un pasillo dividido en 12 celdas, numeradas de 0 a 11. No conoce su posición exacta. Su odometría puede fallar y dispone de un sensor que identifica si en la celda actual hay una marca reflectante, pero puede equivocarse. Tu programa debe mantener una distribución de probabilidad sobre las 12 posibles posiciones.

Mapa conocido de marcas (1 = marca; 0 = sin marca):

```
celda:   0  1  2  3  4  5  6  7  8  9 10 11
marca:   0  0  1  0  1  0  0  1  0  0  1  0
```

El usuario proporciona en cada instante una orden de movimiento $u \in \{-1, 0, +1\}$ y una lectura binaria $z \in \{0, 1\}$. La posición verdadera permanece oculta al algoritmo.

2. Modelo probabilístico obligatorio

Movimiento. Cuando se ordena avanzar o retroceder una celda: probabilidad 0,80 de ejecutar el movimiento; 0,10 de quedarse donde está; 0,10 de moverse una celda adicional en la misma dirección. Para la orden u = 0: probabilidad 0,90 de quedarse; 0,05 de desplazarse a la izquierda; 0,05 a la derecha. En los extremos del pasillo, cualquier desplazamiento fuera del mapa se satura en 0 o 11; conserva la masa de probabilidad.

Sensor. Si existe una marca en la posición real, $P(z = 1 \mid \text{marca} = 1) = 0{,}85$. Si no existe, $P(z = 1 \mid \text{marca} = 0) = 0{,}15$. Las probabilidades para z = 0 son sus complementos.

Estado inicial. El robot empieza con una distribución uniforme sobre las 12 celdas. Posteriormente, añade la opción de pasar una creencia inicial arbitraria válida.

3. Lo que debes implementar

- Representación: vector de 12 probabilidades no negativas cuya suma sea 1.
- Predicción: calcula la distribución tras ejecutar una orden, teniendo en cuenta todos los posibles estados anteriores.
- Corrección: incorpora la observación de la marca aplicando la regla de Bayes y normalizando.
- Estimación: devuelve la celda más probable (MAP), la probabilidad máxima y una medida de incertidumbre (entropía).
- Simulación: genera trayectorias verdaderas y observaciones sintéticas usando exactamente el modelo anterior. Usa una semilla fija para reproducibilidad.

4. Contrato matemático (sin darte la implementación)

Denota por b(j) la probabilidad anterior de estar en j; por T(i | j, u) la probabilidad de acabar en i tras una orden u; y por L(z | i) la probabilidad de observar z en la celda i.

Predicción: b_pred(i) = suma sobre j de T(i | j, u) · b_anterior(j).

Corrección: b_nueva(i) = L(z | i) · b_pred(i) / Z, donde Z = suma sobre k de L(z | k) · b_pred(k).

Entropía: H(b) = -suma sobre i de b(i) · log2(b(i)); define 0 · log2(0) = 0.

La fase de predicción responde a «¿dónde podría estar después de moverse?»; la corrección a «¿cuáles de esas posiciones explican mejor lo observado?». No confundas T(i | j, u) con T(j | i, u).

5. API de referencia (orientativa, no obligatoria)

```
class BayesLocalizer:
    predict(action) -> belief
    update(observation) -> belief
    step(action, observation) -> belief
    estimate() -> (cell, confidence, entropy)
```

Puedes implementarlo en Python con la biblioteca estándar; NumPy es opcional para manejo de arrays, pero no para delegar la actualización bayesiana a una función ya construida. Si prefieres C++17, utiliza std::array<double, 12> y tests unitarios.

6. Pruebas mínimas (obligatorias)

- Normalización: cada predicción y corrección debe sumar 1 (error absoluto inferior a 1e-10) y ninguna entrada debe ser negativa.
- Fronteras: desde la celda 11 y con orden +1, el 100 % de la probabilidad queda en la celda 11. Desde la celda 0 y con -1 ocurre lo mismo.
- Conservación: en ausencia de una observación, la predicción no crea ni destruye masa probabilística.
- Actualización Bayes simple: si prior = [0.2, 0.5, 0.3] y likelihood = [0.1, 0.8, 0.2], el posterior normalizado debe ser aproximadamente [0.0416667, 0.8333333, 0.125]. Esta es una prueba aislada de normalización, independiente del mapa de 12 celdas.
- Entropía: una distribución uniforme sobre 12 celdas debe dar log2(12) bits; una distribución concentrada en una sola celda debe dar 0 bits.

7. Experimentos que tienes que ejecutar

Ejecuta al menos 100 trayectorias de 50 pasos cada una. Inicializa una posición verdadera aleatoria uniforme; en cada paso genera una orden aleatoria entre -1, 0 y +1, simula el movimiento y después la observación. El filtro solo puede ver las órdenes y las lecturas, nunca la posición real.

- Experimento A: filtro completo (movimiento + sensor).
- Experimento B: solo movimiento (sin actualizar por el sensor).
- Experimento C: sensor perfecto en el simulador y en el filtro, manteniendo el mismo modelo de movimiento. Compara y explica el efecto.

8. Métricas y visualizaciones

- Error absoluto: |posición_MAP - posición_real|, promediado en todos los pasos y trayectorias.
- Exactitud: porcentaje de pasos en los que la celda MAP coincide con la real.
- Probabilidad asignada al estado real: promedio de b(posición_real).
- Entropía media: evalúa cuánta incertidumbre declara el modelo.

Genera dos gráficos como mínimo: (1) posición real y posición MAP a lo largo de una trayectoria; (2) mapa de calor de la distribución b(i) por instante. Puedes usar matplotlib solo para visualizar. Presenta una tabla comparando las métricas de A, B y C.

9. Extensiones opcionales, en orden de dificultad

- Bronce: implementa el pasillo circular en lugar de saturar los límites y estudia cómo cambia la ambigüedad.
- Plata: incorpora una segunda modalidad de observación independiente (por ejemplo, un detector de proximidad a una baliza); razona cuándo se pueden multiplicar las verosimilitudes.
- Oro: extiende el filtro a una cuadrícula 2D de 10 x 10, con cuatro acciones cardinales y obstáculos.
- Platino: compara el algoritmo exacto frente a un filtro de partículas que también programes desde cero.

10. Entregables

- Código fuente completo y legible, con funciones separadas de transición, sensor, predicción, corrección y simulación.
- README con instrucciones para ejecutar, dependencias y decisiones de diseño.
- Tests automáticos para los cinco casos mínimos.
- Informe de 2–4 páginas con ecuaciones, gráficos, tabla de experimentos y discusión de fallos.

11. Criterios de evaluación (100 puntos)

Área	Puntos
Modelo de transición y tratamiento de fronteras	20
Corrección bayesiana y estabilidad numérica	20
Simulador reproducible y separación de datos ocultos	15
Pruebas unitarias y casos límite	15
Experimentos, métricas y comparativas	20
Claridad del código, README e informe	10

12. Reglas FHM

Consulta documentación de Python/C++, fórmulas matemáticas y recursos teóricos. No importes implementaciones de filtros bayesianos, HMM o filtros de partículas; no copies una solución completa. Puedes pedir pistas conceptuales, revisión de tests o feedback sobre tu diseño, pero evita solicitar el código final antes de intentarlo.

Condición de éxito: código reproducible que pasa las pruebas, actualiza correctamente la creencia y permite explicar con tus propias palabras por qué una observación reduce (o no) la incertidumbre. No hay una cifra de exactitud prefijada: debes comparar contra el baseline y justificar lo observado.

Pregunta final de defensa: ¿qué sucede si el filtro asume que el sensor acierta el 99 % de las veces cuando realmente solo acierta el 85 %? Diseña una prueba para demostrarlo.